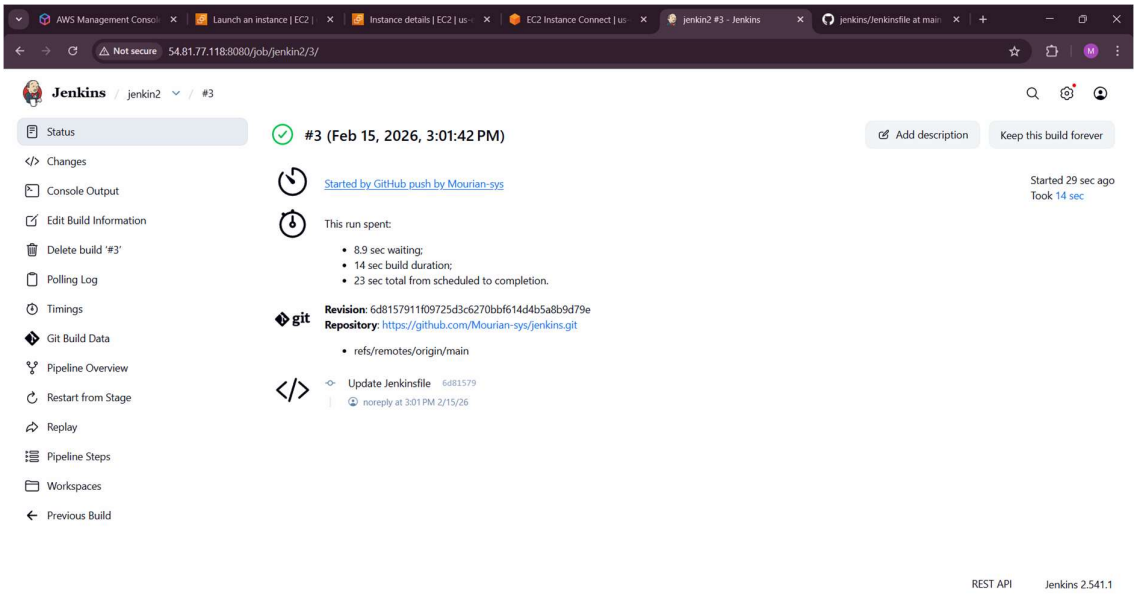
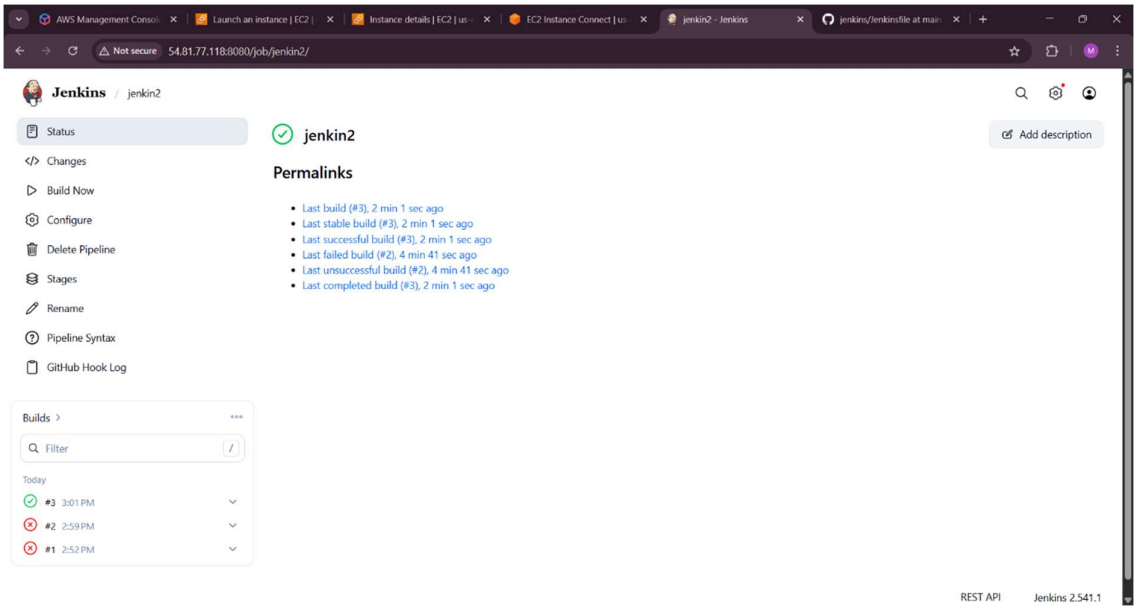
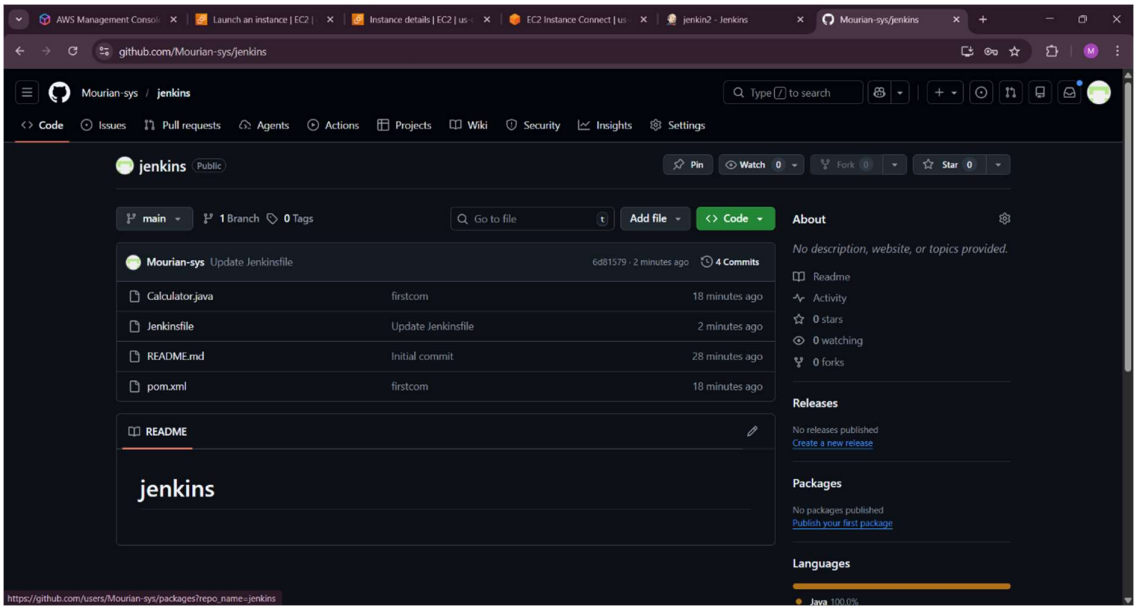
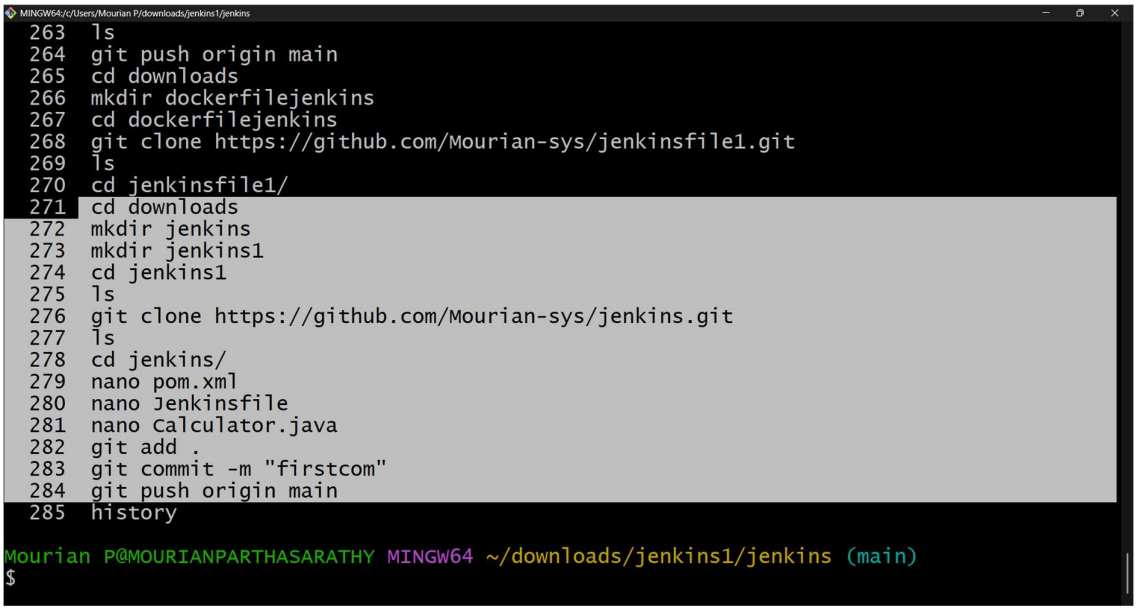
ASSIGNMENT 30

Jenkins with webhook with java project

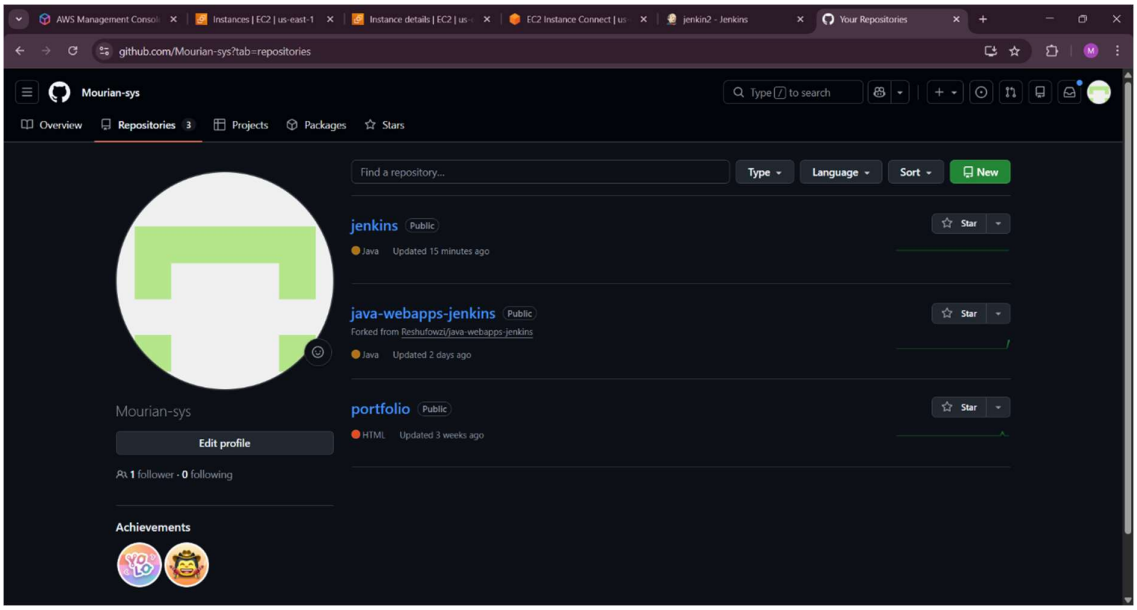





AWS Management Consol
Launch an instance | EC2 |
Instance details | EC2 | us
EC2 Instance Connect | us
jenkin2 - Jenkins
Mourian-sys/jenkins
github.com/Mourian-sys/jenkins
Mourian-sys / jenkins
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security
Insights
Settings
jenkins
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
Mourian-sys Update Jenkinsfile
6d81579 · 2 minutes ago
4 Commits
Calculator.java
firstcom
18 minutes ago
Jenkinsfile
Update Jenkinsfile
2 minutes ago
README.md
Initial commit
28 minutes ago
pom.xml
firstcom
18 minutes ago
README
jenkins
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Languages
Java 100.0%
https://github.com/users/Mourian-sys/packages?repo_name=jenkins


MINGW64:/c/Users/Mourian P/downloads/jenkins1/jenkins
263 ls
264 git push origin main
265 cd downloads
266 mkdir dockerfilejenkins
267 cd dockerfilejenkins
268 git clone https://github.com/Mourian-sys/jenkinsfile1.git
269 ls
270 cd jenkinsfile1/
271 cd downloads
272 mkdir jenkins
273 mkdir jenkins1
274 cd jenkins1
275 ls
276 git clone https://github.com/Mourian-sys/jenkins.git
277 ls
278 cd jenkins/
279 nano pom.xml
280 nano Jenkinsfile
281 nano Calculator.java
282 git add .
283 git commit -m "firstcom"
284 git push origin main
285 history
Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1/jenkins (main)
$

```
MINGW64:/c/Users/Mourian P/downloads/jenkins1/jenkins
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1
$ ls
jenkins/

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1
$ cd jenkins/

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1/jenkins (main)
$ nano pom.xml

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1/jenkins (main)
$ nano Jenkinsfile

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1/jenkins (main)
$ nano Calculator.java

Mourian P@MOURIANPARTHASARATHY MINGW64 ~/downloads/jenkins1/jenkins (main)
$ git add .
warning: in the working copy of 'Calculator.java', LF will be replaced by CRLF the next ti
me Git touches it
warning: in the working copy of 'Jenkinsfile', LF will be replaced by CRLF the next time G
```



Jenkins Setup on EC2 Ubuntu with GitHub Integration

Step 1: Launch EC2 Ubuntu Instance

First, log in to the AWS Management Console. Go to the EC2 service and create a new instance. Choose Ubuntu as the operating system. Select the appropriate instance type such as t2.micro. Configure the security group to allow port 8080 so Jenkins can be accessed from the browser. After launching the instance, copy the public IP address. This IP address will be used to access Jenkins.

Step 2: Open Jenkins in Browser and Complete Initial Setup

After Jenkins is installed on the Ubuntu instance, open a web browser and enter the EC2 public IP address followed by port 8080. Jenkins will open in the browser. During the first login, Jenkins asks for an administrator password. Enter the password to unlock Jenkins.

Next, choose the option to install suggested plugins. Jenkins automatically installs the required basic plugins. After installation, create an admin username and password. Once completed, the Jenkins dashboard will appear.

Step 3: Install Required Plugins Inside Jenkins

From the Jenkins dashboard, go to “Manage Jenkins.” Then click “Plugins.” In the available plugins section, search and install the following plugins:

- Git Plugin (to connect GitHub repositories)
- GitHub Plugin (for webhook integration)
- Pipeline Plugin (to create pipeline jobs)
- Maven Integration Plugin (to support Maven builds)

After selecting the plugins, install them and restart Jenkins if required. These plugins allow Jenkins to automate Java project builds.

Step 4: Configure JDK17 in Jenkins

Go to “Manage Jenkins” and open “Global Tool Configuration.” Scroll to the JDK section and add a new JDK installation. Name it as JDK17. Save the configuration. This step enables Jenkins to use Java 17 for building projects.

Step 5: Configure Maven3 in Jenkins

In the same Global Tool Configuration page, scroll to the Maven section. Add a new Maven installation and name it Maven3. Save the configuration. This allows Jenkins to use Maven as the build tool for Java applications.

Step 6: Add GitHub Credentials for Security

To securely connect Jenkins with GitHub, generate a Personal Access Token from GitHub developer settings. After generating the token, go to Jenkins dashboard. Open “Manage Jenkins” and click “Credentials.” Add new credentials by entering the GitHub username and the generated token as the password. Save the credentials. This ensures secure communication between Jenkins and GitHub.

Step 7: Create a Pipeline Job

From the Jenkins dashboard, click “New Item.” Enter a name for the project and select “Pipeline.” Click OK. In the pipeline configuration section, connect the GitHub repository. Save the configuration. This creates a pipeline job that automates the build process.

Step 8: Configure GitHub Webhook

Open the GitHub repository. Go to “Settings” and click on “Webhooks.” Add a new webhook. Enter the Jenkins URL followed by github-webhook. Save the webhook. This configuration allows GitHub to notify Jenkins automatically whenever code changes are pushed to the repository.

Step 9: Run and Verify the Pipeline

After completing all configurations, the pipeline is ready. When code is pushed to GitHub, the webhook triggers Jenkins automatically. Jenkins fetches the latest code, builds the project, runs tests, and packages the application. The build status is displayed on the Jenkins dashboard. If everything works correctly, Jenkins shows a successful build status.

Conclusion

Jenkins is successfully configured on an EC2 Ubuntu instance. Required plugins such as Git, GitHub, Pipeline, Maven, and JDK are installed inside Jenkins. JDK17 and Maven3 are configured using Global Tool Configuration. GitHub credentials and webhook are added for secure and automatic integration. A pipeline job is created to automate the Java project build process. Now Jenkins automatically builds the project whenever changes are pushed to GitHub.